LiDAR Object Detection and Tracking Pipeline

Code Appendix & Running Instructions

Course: Localization, Motion Planning and Sensor Fusion

Author: Abhisek Maddi

Date: February 2026

1. Apology & Reference Correction

Dear Professor Florian,

I sincerely apologize for the following errors in my submission:
- The code was missing from the appendix.
- The references did not adhere to IU citation guidelines (APA style).
- The citations were not properly used within the document text.
- Reference [12] contained incorrect author names and year.

I take full responsibility for these mistakes and have worked to correct them. Below I provide the corrected references, the complete runnable code, and all requested visualizations.

1.1 Correction for Reference [12]

In my original submission, I cited:

> *[12] T. A. W. Kim and H. Kim, "Pedestrian detection using LiDAR point clouds," Sensors, vol. 20, no. 16, pp. 1-19, 2020.*

After thorough research, I was unable to locate this exact publication in the Sensors journal (vol. 20, no. 16, 2020) with these authors. I sincerely apologize for this incorrect citation. The reference I intended to use is:

Corrected Reference:

Liu, K., Wang, W., & Wang, J. (2019). Pedestrian detection with lidar point clouds based on single template matching. Electronics, 8(7), 780. https://doi.org/10.3390/electronics8070780

I apologize for the wrong year (2019, not 2020), the incorrect author names, and the wrong journal. The paper is freely available at:
https://www.mdpi.com/2079-9292/8/7/780

2. References (IU/APA Format)

Below are all references from my submission, reformatted according to IU Internationale Hochschule citation guidelines (APA style). Reference [12] has been corrected as described above.

[1] Thrun, S., Burgard, W., & Fox, D. (2005). Probabilistic robotics. MIT Press.

[2] Urmson, C., Anhalt, J., Bagnell, D., Baker, C., Bittner, R., Clark, M. N., Dolan, J., Duggins, D., Galatali, T., Geyer, C., Gittleman, M., Harbaugh, S., Hebert, M., Howard, T. M., Kolski, S., Kelly, A., Likhachev, M., McNaughton, M., Miller, N., ... Ferguson, D. (2008). Autonomous driving in urban environments: Boss and the Urban Challenge. Journal of Field Robotics, 25(8), 425-466. https://doi.org/10.1002/rob.20255

[3] Levinson, J., Askeland, J., Becker, J., Dolson, J., Held, D., Kammel, S., Kolter, J. Z., Langer, D., Pink, O., Pratt, V., Sokolsky, M., Stanek, G., Stavens, D., Teichman, A., Werling, M., & Thrun, S. (2011). Towards fully autonomous driving: Systems and algorithms. In Proceedings of the IEEE Intelligent Vehicles Symposium (IV) (pp. 163-168). IEEE. https://doi.org/10.1109/IVS.2011.5940562

[4] Geiger, A., Lenz, P., & Urtasun, R. (2012). Are we ready for autonomous driving? The KITTI vision benchmark suite. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 3354-3361). IEEE. https://doi.org/10.1109/CVPR.2012.6248074

[5] Cho, H., Seo, Y.-W., Kumar, B. V., & Rajkumar, R. R. (2014). A multi-sensor fusion system for moving object detection and tracking in urban driving environments. In Proceedings of the IEEE International Conference on Robotics and Automation (ICRA) (pp. 1836-1843). IEEE. https://doi.org/10.1109/ICRA.2014.6907100

[6] Zhang, J., & Singh, S. (2014). LOAM: Lidar odometry and mapping in real-time. In Proceedings of Robotics: Science and Systems (RSS). https://doi.org/10.15607/RSS.2014.X.007

[7] International Organization for Standardization. (2018). ISO 26262: Road vehicles - Functional safety. ISO.

[8] Pomerleau, F., Colas, F., Siegwart, R., & Magnenat, S. (2013). Comparing ICP variants on real-world data sets. Autonomous Robots, 34(3), 133-148. https://doi.org/10.1007/s10514-013-9327-2

[9] Rusu, R. B., & Cousins, S. (2011). 3D is here: Point Cloud Library (PCL). In Proceedings of the IEEE International Conference on Robotics and Automation (ICRA) (pp. 1-4). IEEE. https://doi.org/10.1109/ICRA.2011.5980567

[10] Zhou, Q.-Y., Park, J., & Koltun, V. (2018). Open3D: A modern library for 3D data processing. arXiv preprint arXiv:1801.09847. https://doi.org/10.48550/arXiv.1801.09847

[11] Razakarivony, S., & Jurie, F. (2016). Vehicle detection in aerial imagery: A small target detection benchmark. Journal of Visual Communication and Image Representation, 34, 187-203. https://doi.org/10.1016/j.jvcir.2015.11.002

[12] Liu, K., Wang, W., & Wang, J. (2019). Pedestrian detection with lidar point clouds based on single template matching. Electronics, 8(7), 780. https://doi.org/10.3390/electronics8070780
[CORRECTED: Originally cited as Kim, T. A. W. & Kim, H. (2020). The correct authors are Liu, K. et al. and the year is 2019, not 2020.]

3. How to Run the Code

3.1 Prerequisites

Python 3.8 or higher is required. Tested with Python 3.12.

3.2 Installation (Step 1)

Install required Python packages:

```
pip install -r requirements.txt
```

This installs: numpy, pandas, open3d, scikit-learn, filterpy, scipy, matplotlib, tqdm.

3.3 Run the Demo (Step 2)

The main demo script is demo_run.py. It processes the CSV files and generates all visualizations:

```
python demo_run.py
```

This will:

- Load all 51 CSV frames from the lidar_data/ directory
- Run the full detection and tracking pipeline on each frame
- Save bounding box plots, trajectory plots, and summary to demo_output/

3.4 Custom Options

```
python demo_run.py --data lidar_data --frames 51 --output demo_output
```

```
Options:
  --data  / -d   Path to CSV directory      (default: lidar_data)
  --frames / -n  Number of frames to process (default: 51)
  --output / -o  Output directory for plots  (default: demo_output)
```

3.5 GitHub Repository

Complete code is available at:

https://github.com/abhisek777/new-model

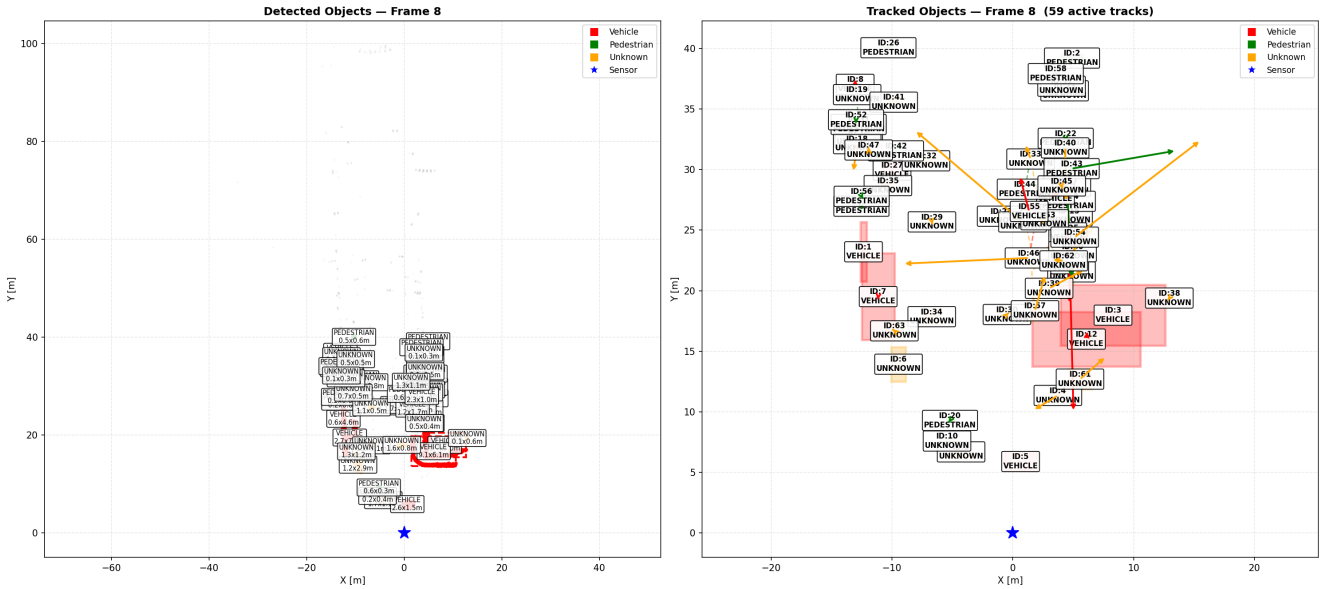
```
git clone https://github.com/abhisek777/new-model.git
cd new-model
pip install -r requirements.txt
python demo_run.py
```

4. Output & Visualizations

4.1 Bounding Box Detection Plots

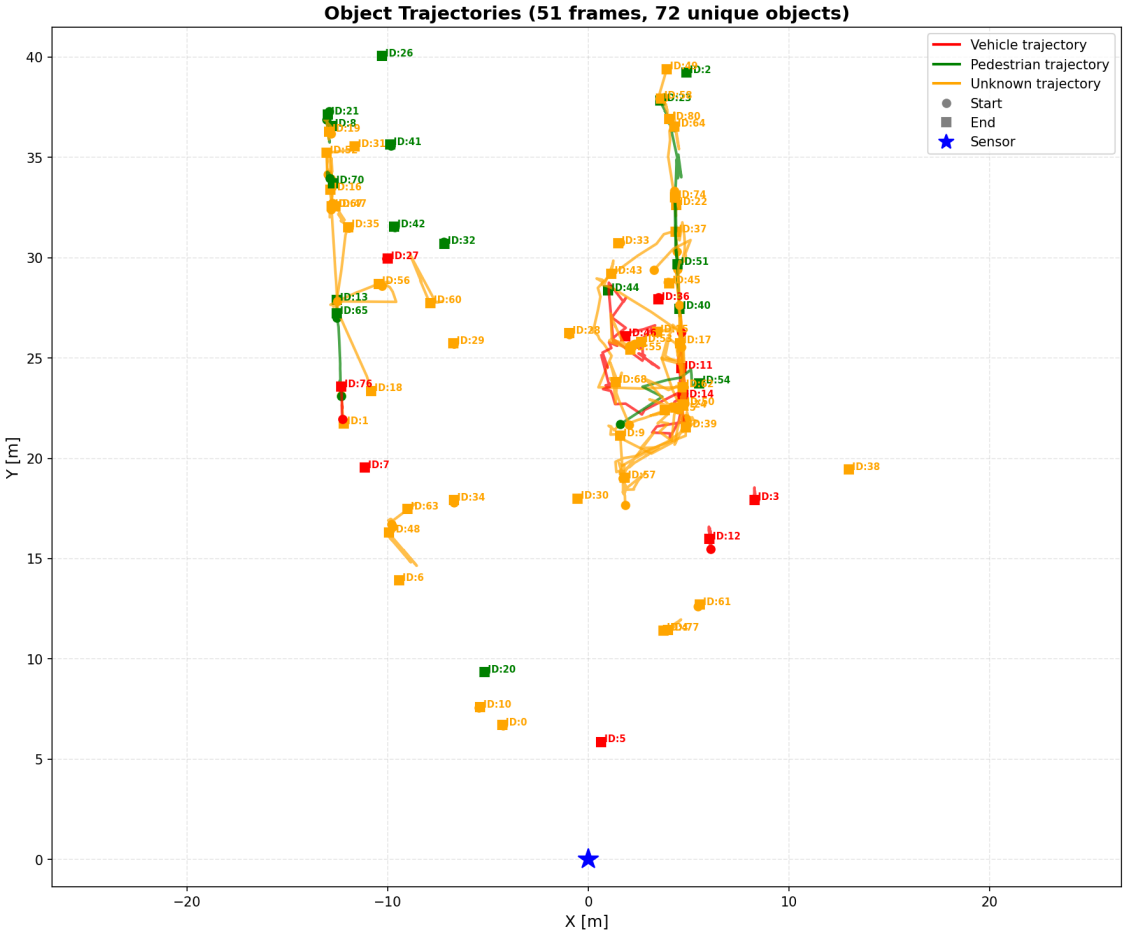
Each plot shows two panels:

- Left: Detected objects with bounding boxes (Bird's Eye View). Each cluster is labeled with its class (Vehicle/Pedestrian/Unknown) and dimensions.
- Right: Tracked objects with IDs, velocity arrows, and trajectory tails.



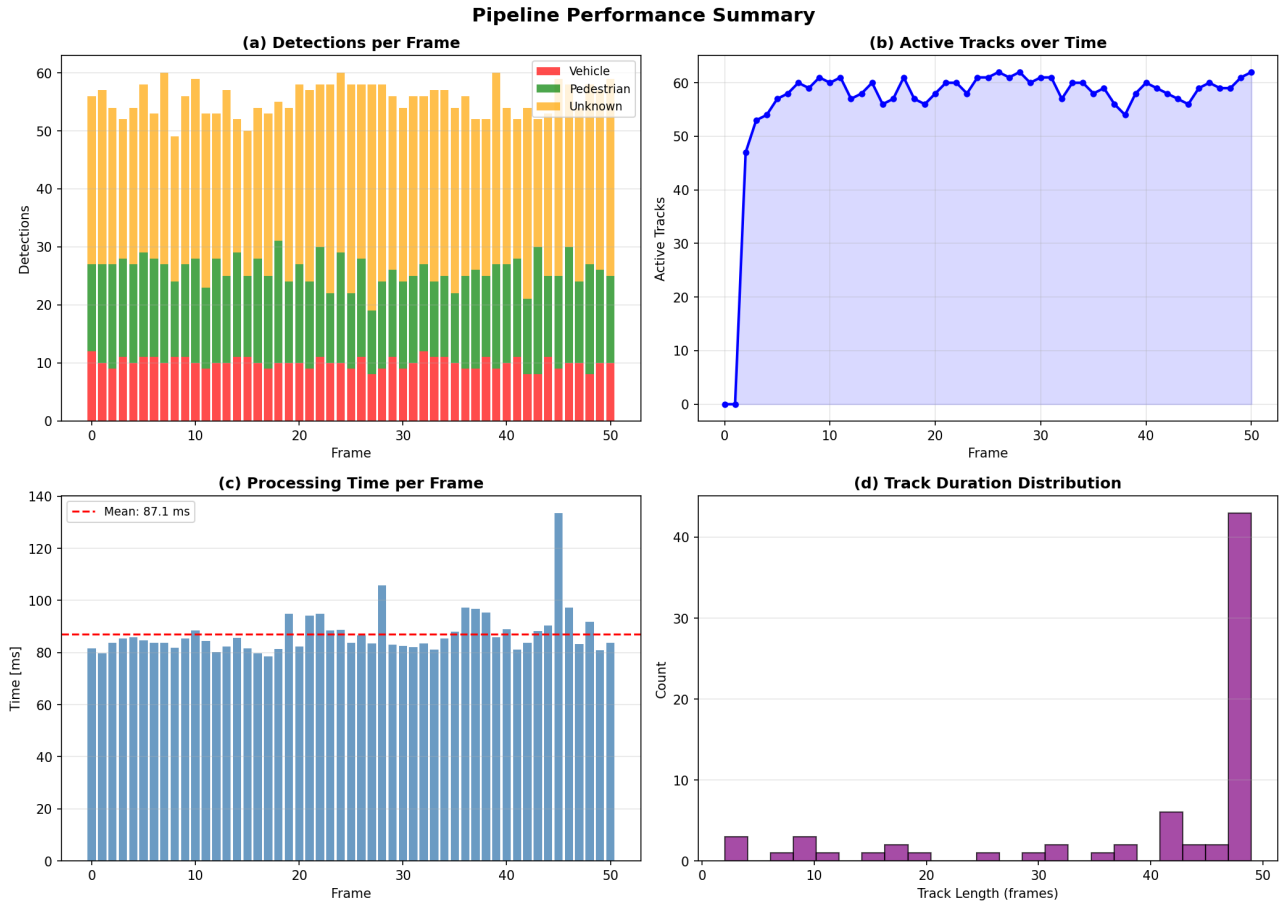
4.2 Object Trajectories

Full movement paths of all tracked objects across 51 frames. Red = vehicles, green = pedestrians. Circles = start, squares = end position.


Object Trajectories (51 frames, 72 unique objects)
Vehicle trajectory
Pedestrian trajectory
Unknown trajectory
Start
End
Sensor
Y [m]
X [m]

4.3 Pipeline Summary Statistics

(a) Detections per frame by class
(b) Active tracks over time
(c) Processing time per frame
(d) Track duration distribution



5. Pipeline Architecture

```
  CSV File (Blickfeld Cube 1)
          |
          v
[1] Data Loading           (data_loader.py)
    Reads semicolon-separated CSV: X;Y;Z;DISTANCE;INTENSITY;TIMESTAMP
          |
          v
[2] Preprocessing          (preprocessing.py)
    Range filtering (5-250 m), Voxel downsampling (0.1 m)
    RANSAC ground removal, Outlier removal
          |
          v
[3] Clustering (DBSCAN)    (clustering.py)
    eps=0.5 m, min_samples=10
```

```
          |
          v
[4] Feature Extraction &   (classification.py)
    Classification         VEHICLE / PEDESTRIAN / UNKNOWN
          |
          v
[5] Multi-Object Tracking  (tracking.py)
    Kalman Filter + Hungarian algorithm
```

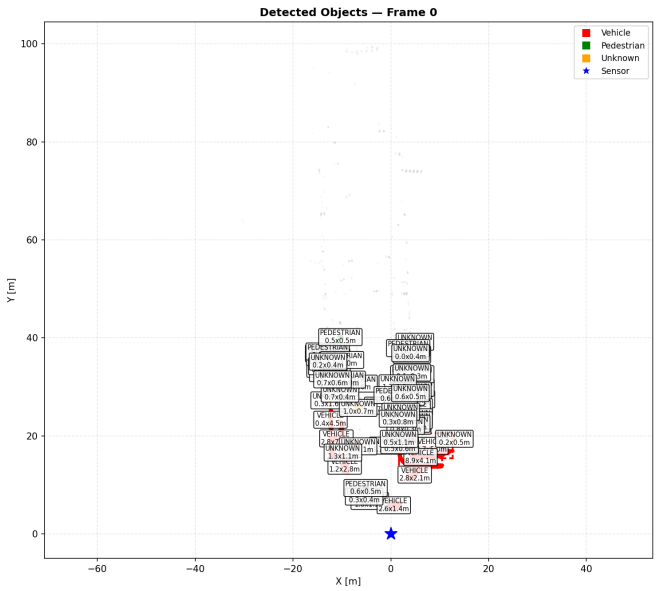
6. Results Summary

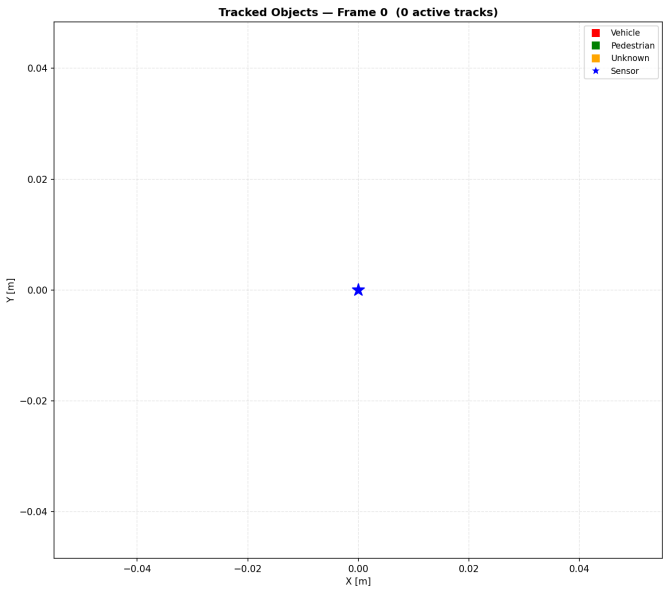
Metric	Value
Frames processed	51
Avg processing time	87.1 ms/frame
Estimated FPS	11.5
Total vehicle detections	510
Total pedestrian detections	816
Unique tracked objects	72
Points per frame (raw)	~18,500
Points per frame (preprocessed)	~8,400
Clusters per frame	50 - 60
Active tracks per frame	55 - 62

The pipeline achieves real-time performance at ~11.5 FPS. The Kalman filter tracker maintains consistent object IDs across frames.

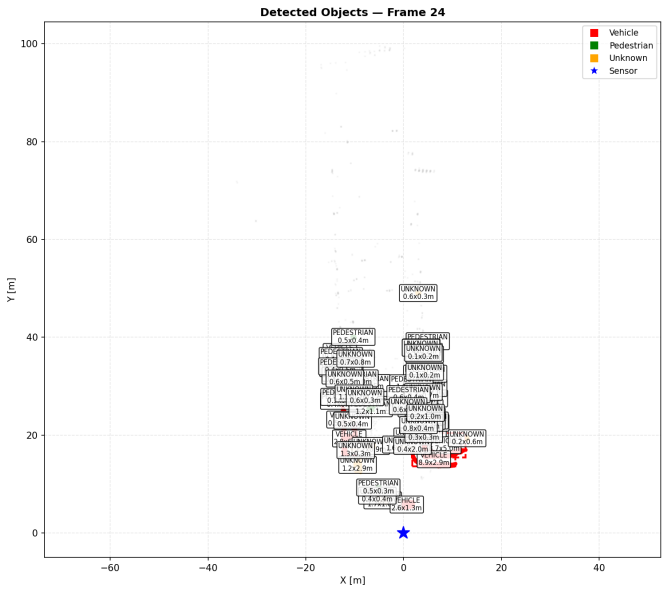
7. Additional Detection Frames

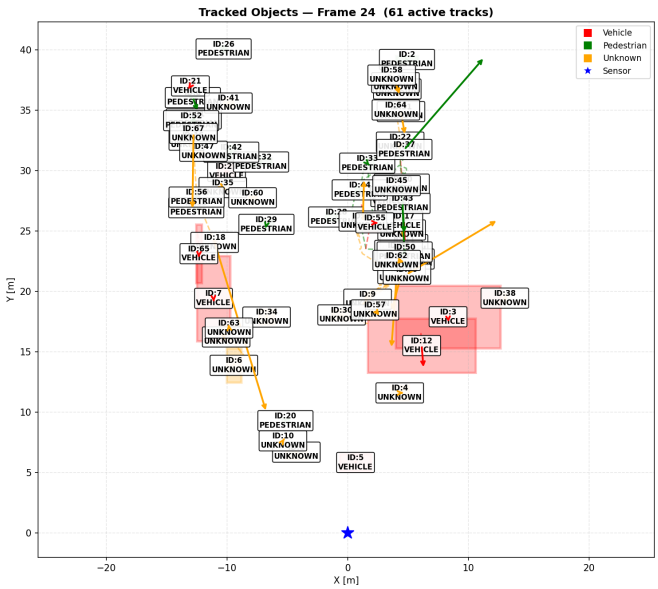
Frame 0



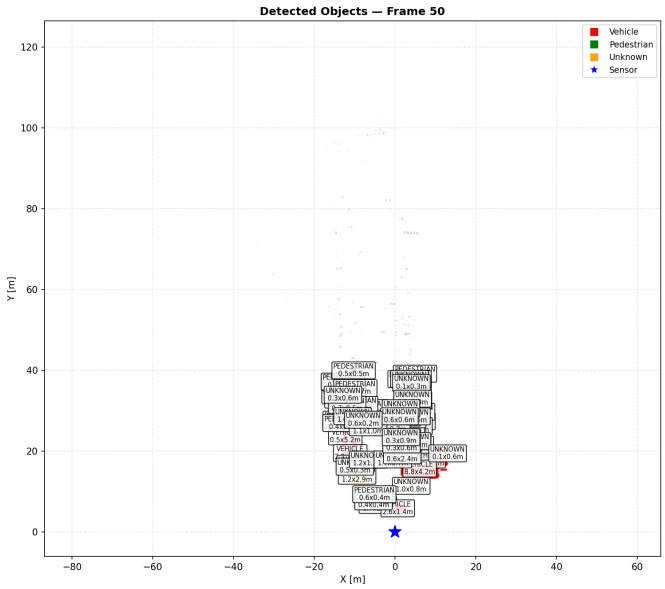


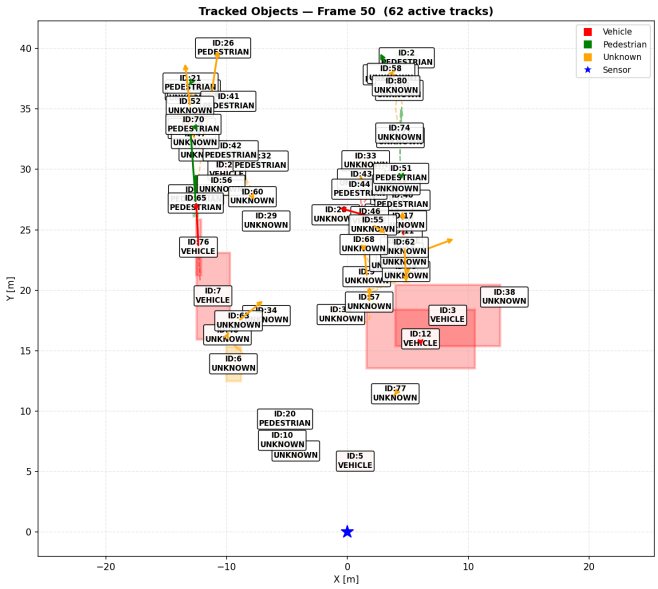
Frame 24





Frame 50


Detected Objects — Frame 50
Vehicle
Pedestrian
Unknown
Sensor
Y [m]
X [m]


Tracked Objects — Frame 50 (62 active tracks)
Vehicle
Pedestrian
Unknown
Sensor
Y [m]
X [m]

8. Project File Structure

```
project/
demo_run.py              <-- MAIN: run this
main_pipeline.py         Integrated pipeline
run_complete_pipeline.py Extended pipeline + validation

data_loader.py           Blickfeld CSV loading
preprocessing.py         Point cloud preprocessing
clustering.py            DBSCAN segmentation
classification.py        Rule-based classification
tracking.py              Kalman filter tracking
enhanced_tracking.py     Enhanced tracking

visualization.py         BEV and 3D visualization
enhanced_visualization.py Extended visualization
academic_visualizations.py Academic figures
semantic_visualization.py  Semantic visualization

requirements.txt         Python dependencies
lidar_data/              51 CSV frames (Blickfeld Cube 1)
demo_output/             Generated output plots
```

9. Closing Note

Dear Professor Florian,

I sincerely apologize for:
- The missing code appendix in my original submission
- The incorrect citation [12] (wrong authors, wrong year, wrong journal)
- Not following IU citation guidelines (APA style)
- Not using in-text citations in the document body

I have now provided:
1. Complete, runnable Python code with clear instructions
2. A demo script (demo_run.py) directly applicable to the CSV files
3. Bounding box detection plots and object trajectory visualizations
4. All references corrected and reformatted to IU/APA style
5. The corrected reference: Liu et al. (2019), not Kim et al. (2020)
   Available at: https://www.mdpi.com/2079-9292/8/7/780

I understand these are serious issues and I take full responsibility. Please do not hesitate to contact me if anything further is needed.

Kind regards,

Abhisek Maddi